

Operações com bits



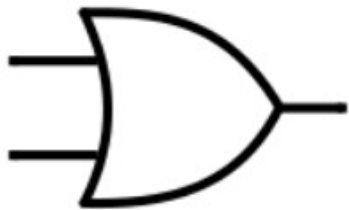
Variável booleana

V	0
	1

Nível lógico 0	Nível lógico 1
Falso	Verdadeiro
Desligado	Ligado
Baixo	Alto
Não	Sim
Chave aberta	Chave fechada
binário	

Operações booleanas

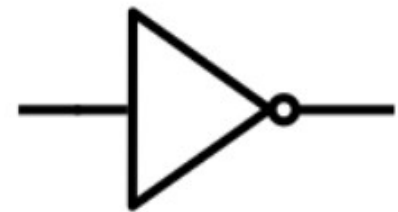
Operação **OU**



Operação **E**

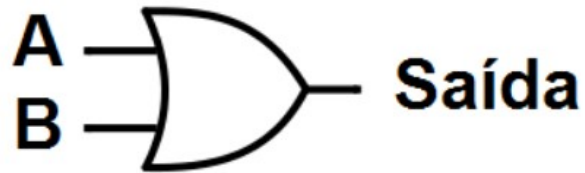


Operação **Não**



Operações booleanas

Operação OU



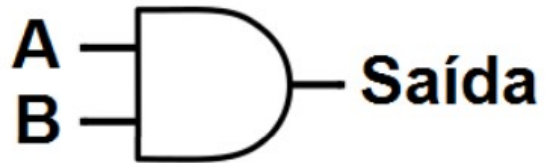
$$\text{Saída} = A + B$$

A	B	Saída
0	0	0
0	1	1
1	0	1
1	1	1



Operações booleanas

Operação E

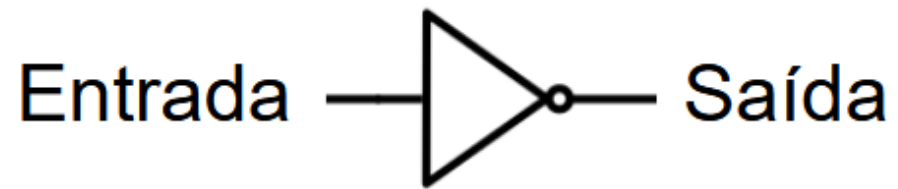


$$\text{Saída} = A.B$$

A	B	Saída
0	0	0
0	1	0
1	0	0
1	1	1

Operações booleanas

Operação **Não**

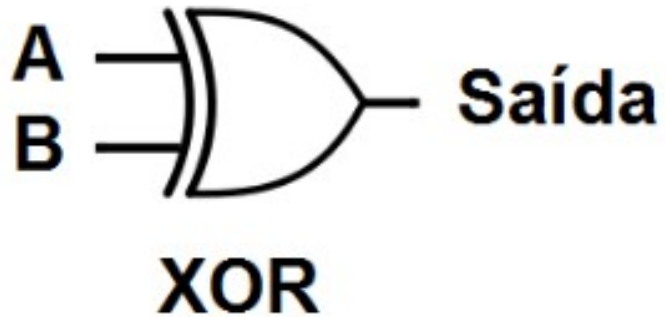


Entrada	Saída
0	1
1	0



Operações booleanas

Operação OU Exclusivo

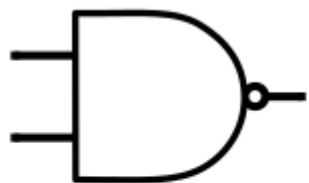


$$\text{Saída} = A \oplus B$$

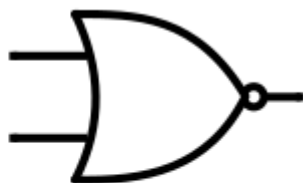
A	B	XOR
0	0	0
0	1	1
1	0	1
1	1	0

Operações booleanas

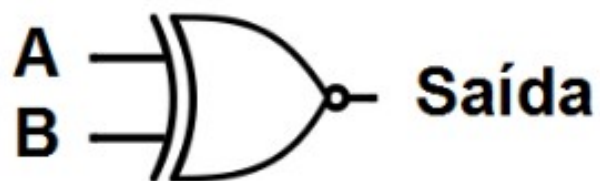
Portas com inversão



Porta NE



Porta NOU



Não XOR



Operações lógicas em linguagem C



Operações lógicas

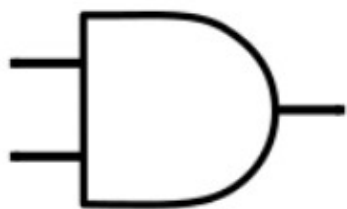
- Os operadores relacionais fazem comparações.
 - O resultado de uma comparação é verdadeiro (1) ou falso (0).
- A linguagem C não define um tipo lógico, somente

Operador	Descrição	Operador	Descrição
>	Maior	<	Menor
>=	Maior ou igual	<=	Menor ou igual
==	Igual	!=	Diferente

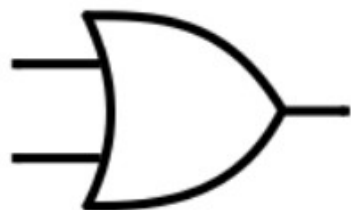
Operações lógicas

- Os operadores lógicos são:

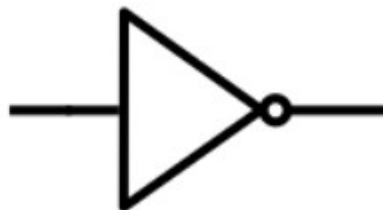
- Lógico E: indicado por **&&**
- Lógico OU: indicado por **||**
- Lógico Não: indicado por **!**



E



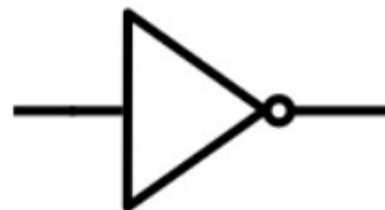
OU



Não



Operações lógicas



• Exemplos:

- `exp1 && exp2`: é verdadeira se as duas expressões forem verdadeiras.
- `exp1 || exp2`: é verdadeira se uma ou as duas expressões forem verdadeiras.
- `! exp1`: é verdadeira se `exp1` for falsa e vice-versa.

Operações bitwise em linguagem C

(bit a bit)



Operação de negação

- Declaração

```
char A = 8;  
// A = 00001000
```

- Operação lógica

```
r = !A;  
// r = 0
```

- Operação bitwise

```
r = ~A;  
// A = 00001000  
// r = 11110111
```



Operação E

- Declaração

```
char A = 8;  
// A = 00001000  
char B = 5;  
// B = 00000101
```

- Operação lógica

```
result = A && B;  
// result = 1
```

- Operação bitwise

```
result = A & B;  
// A = 00001000  
// B = 00000101  
// r = 00000000
```



Operação OU

- Declaração

```
char A = 8;  
// A = 00001000  
char B = 5;  
// B = 00000101
```

- Operação lógica

```
result = A || B;  
// r = 1
```

- Operação bitwise

```
result = A | B;  
// A = 00001000  
// B = 00000101  
// r = 00001101
```



Operação XOR

- Declaração

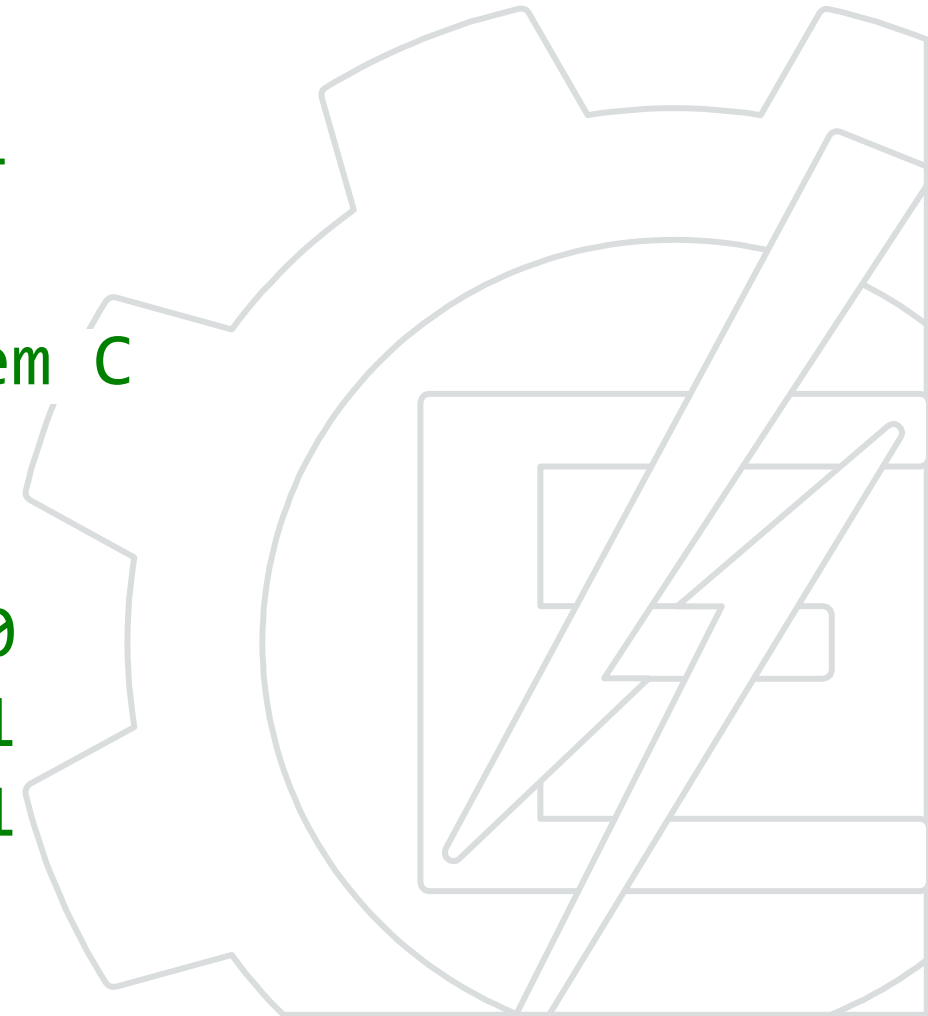
```
char A = 8;  
// A = 00001000  
char B = 5;  
// B = 00000101
```

- Operação lógica

```
// não existe em C
```

- Operação bitwise

```
r = A ^ B;  
// A = 00001000  
// B = 00000101  
// r = 00001101
```



Deslocamento de bits

A operação shift desloca os bits.

- Para a esquerda operador "<<".
- Para direita operador ">>".

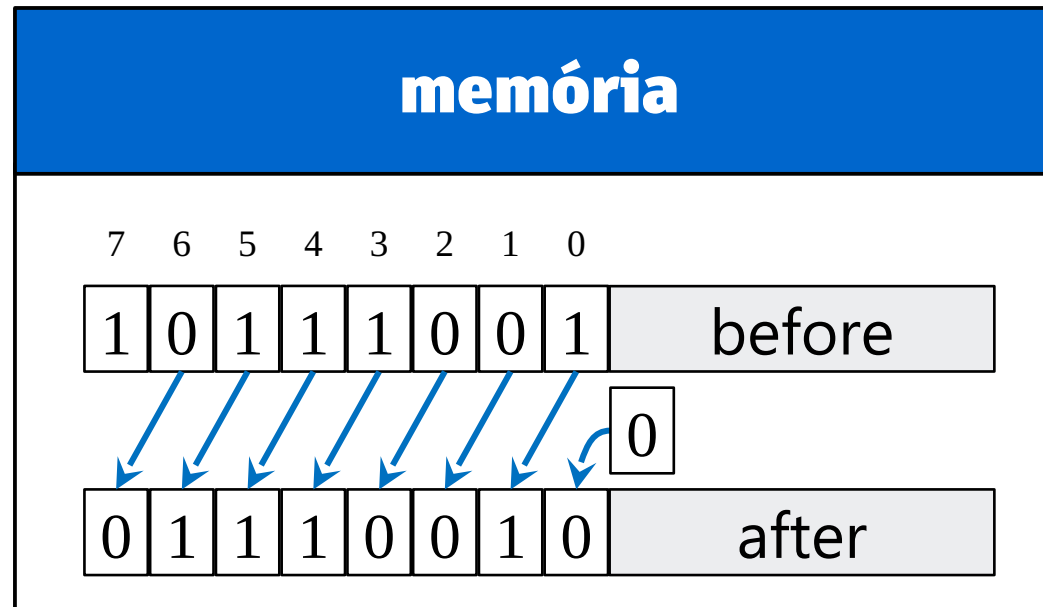
É necessário indicar quantas casas serão deslocadas



Deslocamento de bits para esquerda

```
unsigned char before;  
unsigned char after;  
before = 0xB9;  
after = before << 1;  
//after = 0x72
```

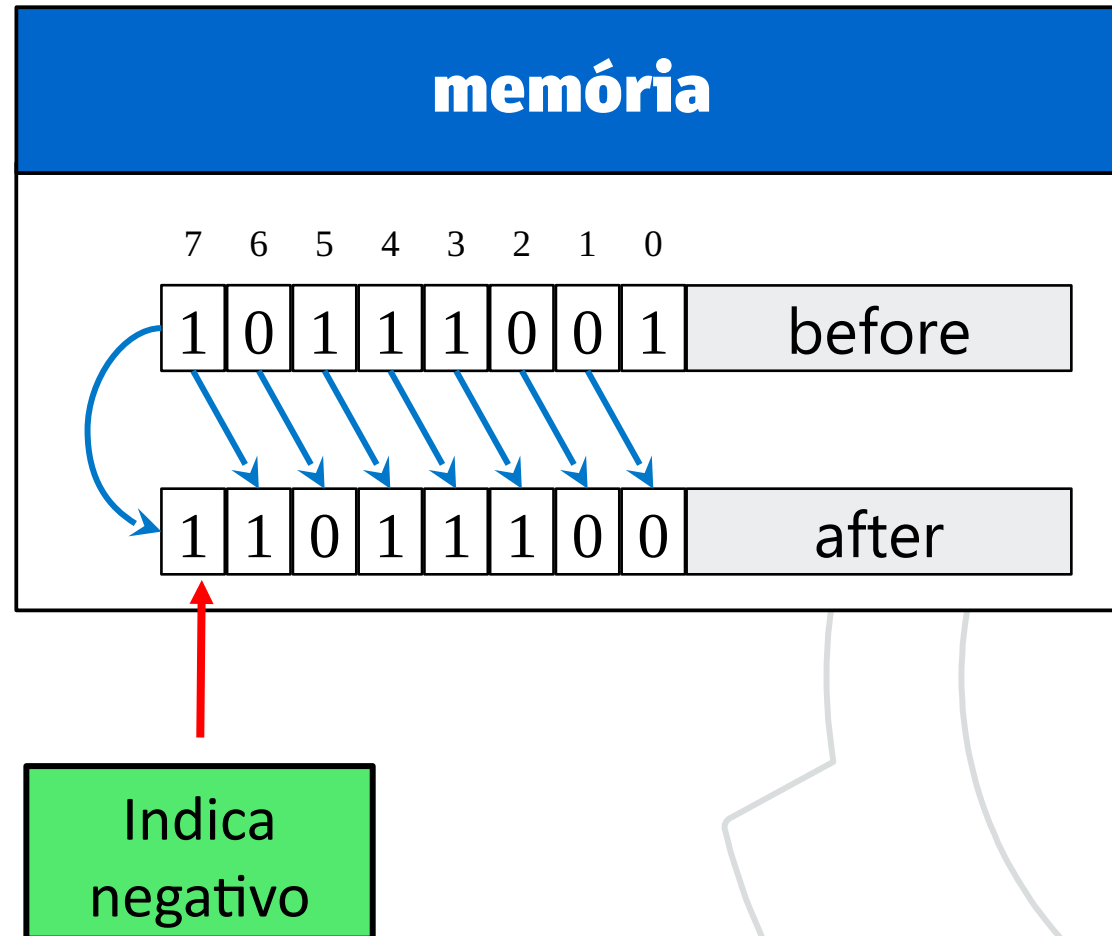
Deslocar 1



Deslocamento de bits para direita

```
char before;  
char after;  
before = 0xB9;  
after = before >> 1;  
//after = 0xDC
```

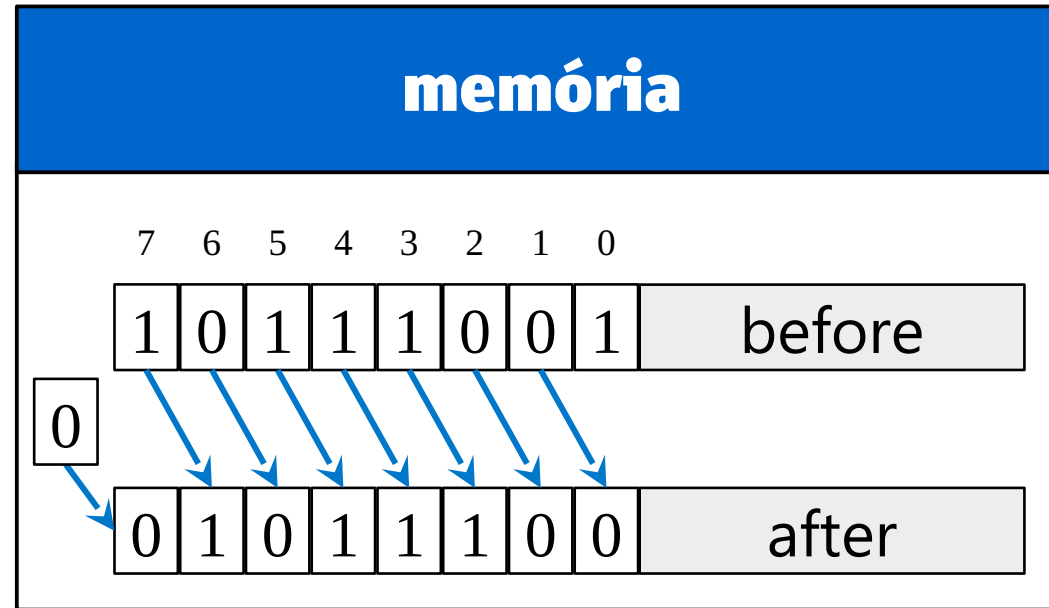
Deslocar 1



Deslocamento de bits para direita

```
unsigned char before;  
unsigned char after;  
before = 0xB9;  
after = before >> 1;  
//after = 0x5C
```

Se fosse **signed**
repetiria o bit 7 que é 1



Operações com um único bit

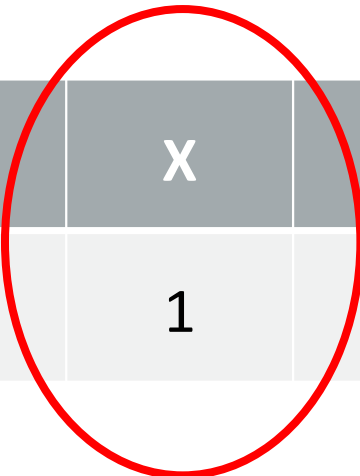


Como ligar um bit

- Criar uma "máscara" cheia de 0 (zeros) e com 1 (um) apenas na posição desejada.

Fazer uma operação OR bit a bit entre a variável e a máscara.

Posição	N	...	X+1	X	X-1	...	0
Valor	0	...	0	1	0	...	0



Ligando um bit

//Passo a passo

```
char bit = 2;
```

```
char mascara, arg = 0xB9;
```

```
mascara = 1 << bit;
```

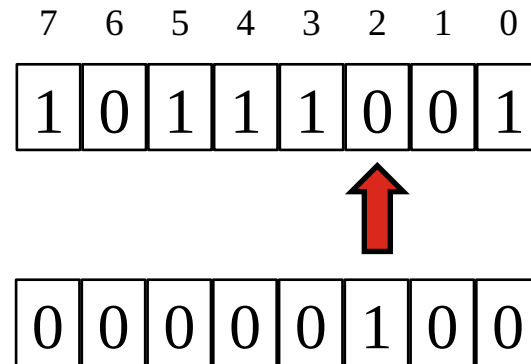
```
arg = arg | mascara;
```

//Fazendo em uma linha

```
arg |= (1<<bit);
```

//Utilizando define

```
#define BitSet(arg,bit) ((arg) |= (1<<bit))
```

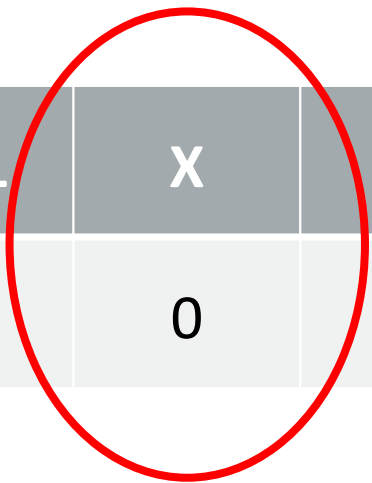


Como desligar um bit

- Criar uma "máscara" cheia de 1 (uns) e com 0 (zero) apenas na posição desejada.

Fazer uma operação AND bit a bit entre a variável e a máscara.

Posição	N	...	X+1	X	X-1	...	0
Valor	1	...	1	0	1	...	1



Desligando um bit

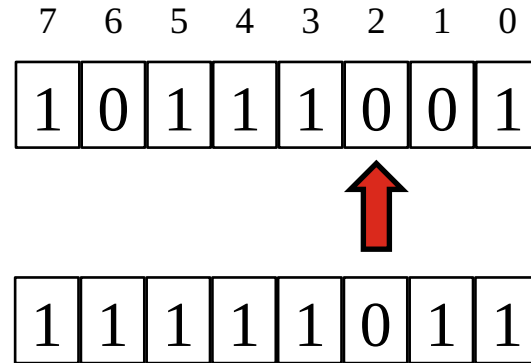
//Passo a passo

```
char bit = 2;
```

```
char mascara, arg = 0xB9;
```

```
mascara = 1 << bit;
```

```
arg = arg & ~mascara;
```



//Uma linha

```
arg &= ~(1<<bit);
```

//Com define

```
#define BitClr(arg,bit) ((arg) &= ~(1<<bit))
```

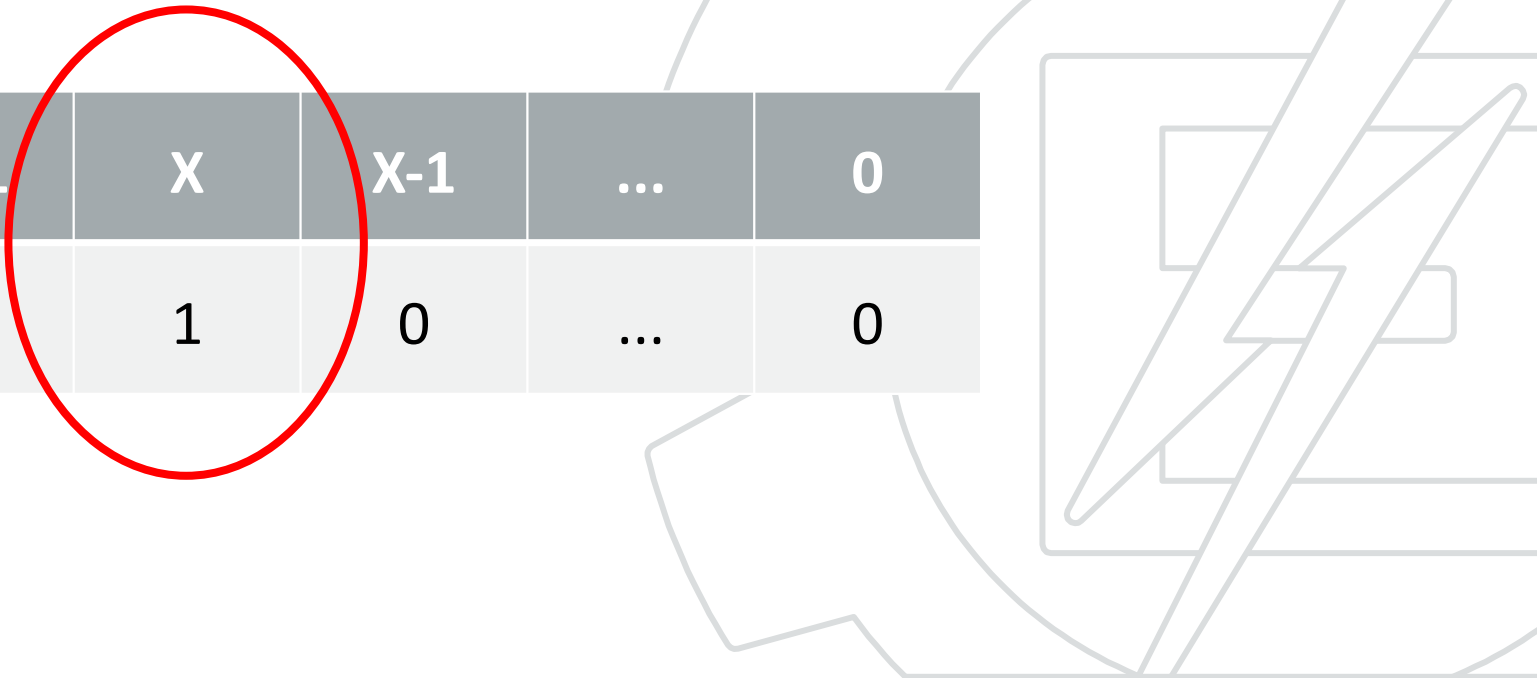


Como trocar o valor de um bit

- Criar uma "máscara" cheia de 0 (zeros) e com 1 (um) apenas na posição desejada.

Fazer uma operação XOR bit a bit entre a variável e a máscara.

Posição	N	...	X+1	X	X-1	...	0
Valor	0	...	0	1	0	...	0



Trocando o valor de um bit

//Passo a passo

```
char bit = 2;
```

```
char mascara, arg = 0xB9;
```

```
mascara = 1 << bit;
```

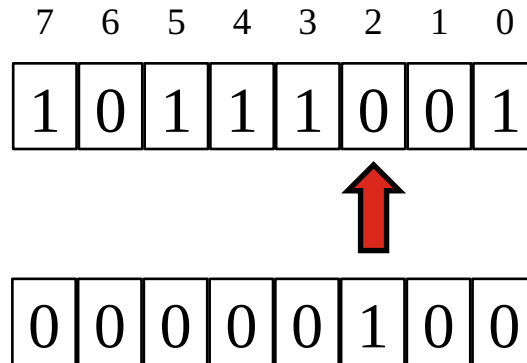
```
arg = arg ^ mascara;
```

//Uma linha

```
arg ^= (1<<bit);
```

//Com define

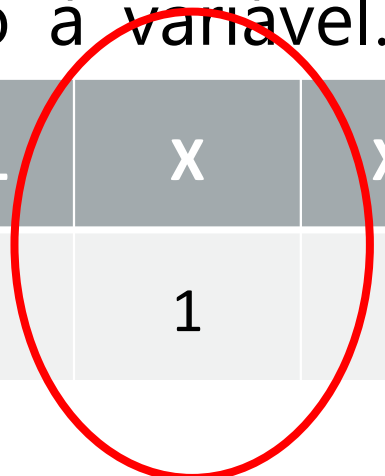
```
#define BitFlp(arg,bit) ((arg) ^= (1<<bit))
```



Como verificar o estado de um bit

- Criar uma "máscara" cheia de 0 (zeros) e com 1 (um) apenas na posição desejada.
- Fazer uma operação AND bit a bit entre a variável e a máscara.
- Não atribuir o resultado à variável.

Posição	N	...	X+1	X	X-1	...	0
Valor	0	...	0	1	0	...	0



Verificando o estado de um bit

//Passo a passo

```
char bit = 2;
```

```
char mascara, arg = 0xB9;
```

```
mascara = 1 << bit;
```

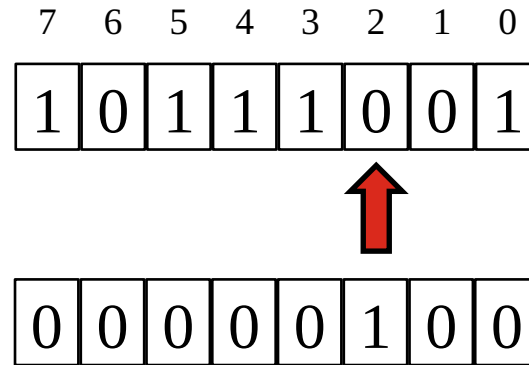
```
if (arg & mascara) { }
```

//Uma linha

```
if (arg & (1<<bit)) { }
```

//Com define

```
#define BitTst(arg,bit) ((arg) & (1<<bit))
```



Verificando o estado de um bit

Cuidado



Imagem: www.graphicsfactory.com

Atenção

Verdadeiro é qualquer valor diferente de zero

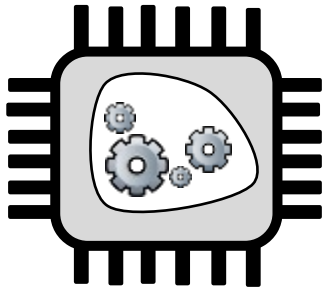
Falso é zero

Introdução a microcontroladores



Microprocessador

Microprocessador



roteiro

comando 1
comando 2
comando 3
...

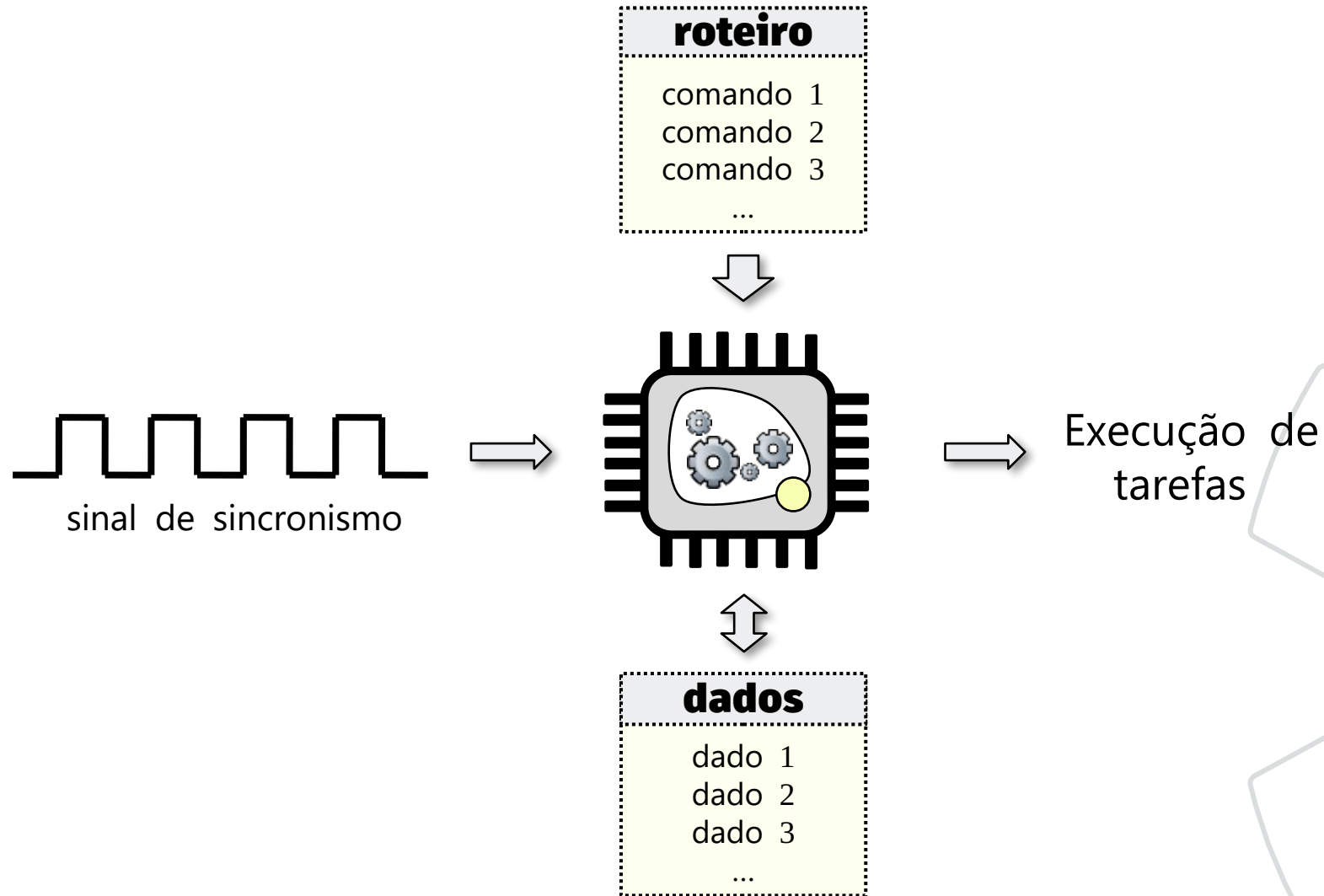
Circuitos digitais dedicados

Propósito geral

Operação programável

O uso de um microprocessador é que define que um sistema é embarcado

Modelo de aplicação



Princípio de operação

Comando 1
Comando 2
Comando 3
Comando 4
Comando 5
Comando 6
...

Initializing PORTA

```
CLRF    PORTA      ;Initialize PORTA by
                   ;setting
                   ;output data latches
MOVLW   0x07        ;Turn comparators off and
MOVWF   CMCON       ;enable pins for I/O
                   ;functions

BCF     STATUS, RP1
BSF     STATUS, RP0 ;Select Bank1
MOVLW   0x1F        ;Value used to initialize
                   ;data direction
MOVWF   TRISA       ;Set RA<4:0> as inputs
                   ;TRISA<5> always
                   ;read as '1'.
                   ;TRISA<7:6>
                   ;depend on oscillator
                   ;mode
```

Princípio de operação

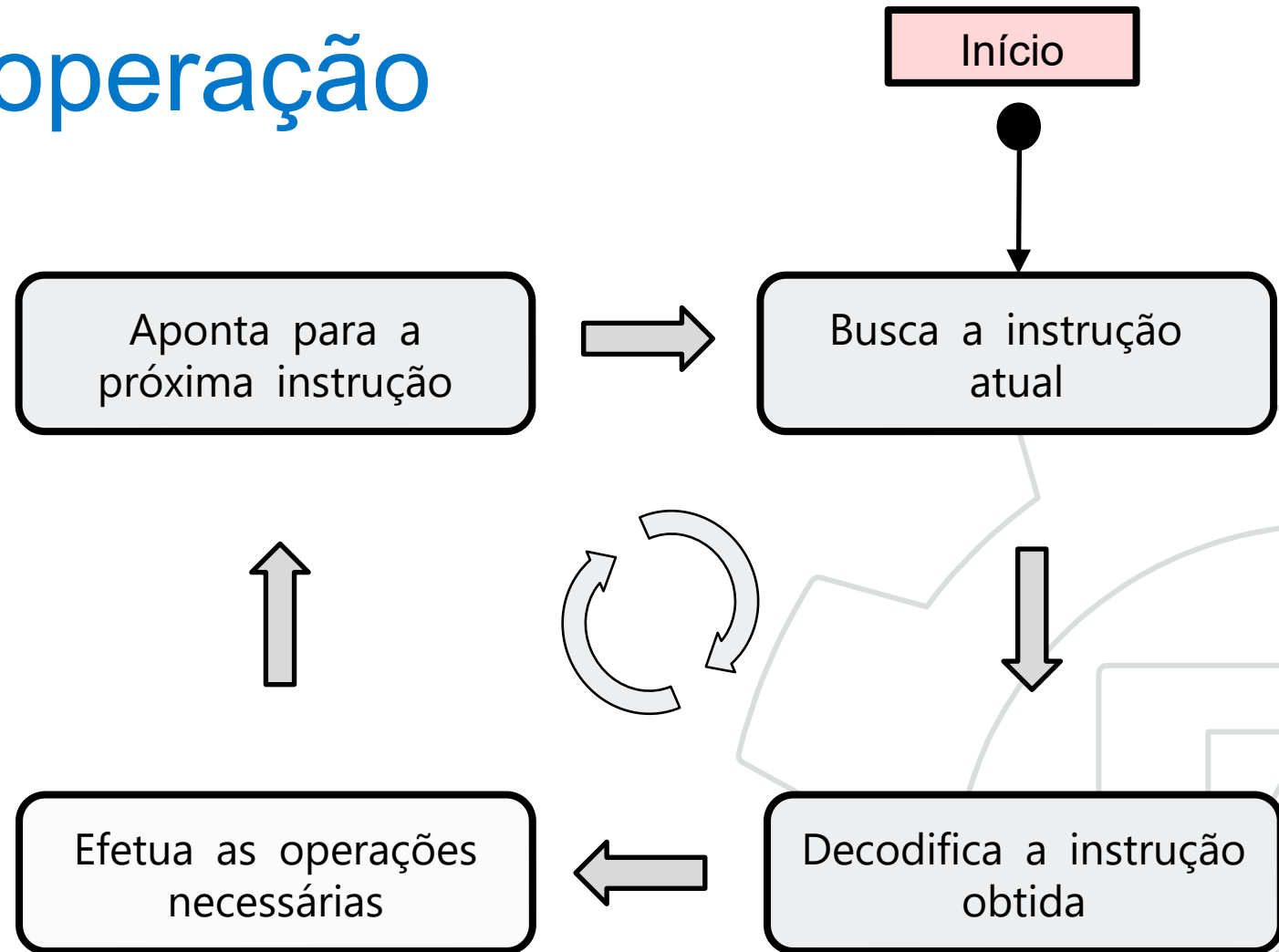
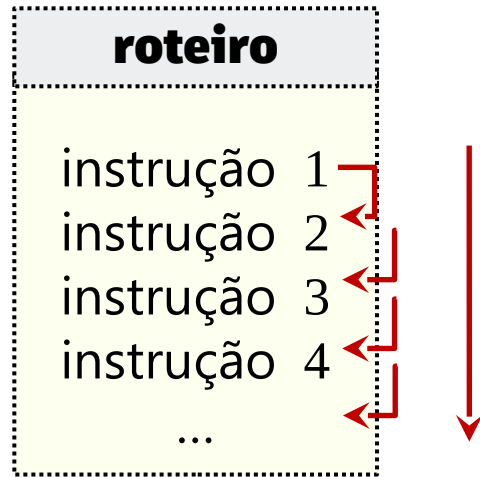
Initializing PORTA

```
CLRF    PORTA      ;Initialize PORTA by
                  ;setting
                  ;output data latches
MOVLW   0x07        ;Turn comparators off and
MOVWF   CMCON       ;enable pins for I/O
                  ;functions

BCF     STATUS, RP1
BSF     STATUS, RP0 ;Select Bank1
MOVLW   0x1F        ;Value used to initialize
                  ;data direction
MOVWF   TRISA       ;Set RA<4:0> as
                  ;TRISA<5> always
                  ;read as '1'.
                  ;TRISA<7:6>
                  ;depend on osc
                  ;mode
```

```
*R PC=FFFF SP=FF A=FF B=FF PSW=FF DPTR=FFFF
R0=FF R1=FF R2=FF R3=FF R4=FF R5=FF R6=FF R7=FF
*D 2050,2100
2050 01 02 04 08 10 20 40 80 55 AA 33 01 02 04 08 10 ..... @.U.3.....
2060 20 40 80 55 AA 33 01 02 04 08 10 20 40 80 55 AA @.U.3..... @.U.
2070 33 01 02 04 08 10 20 40 80 55 AA 33 01 02 04 08 3..... @.U.3....
2080 10 20 40 80 55 AA 33 01 02 04 08 10 20 40 80 55 . @.U.3..... @.U
2090 AA 33 01 02 04 08 10 20 40 80 55 AA 33 01 02 04 .3..... @.U.3...
20A0 08 10 20 40 80 55 AA 33 01 02 04 08 10 20 40 80 .. @.U.3..... @.
20B0 55 AA 33 01 02 04 08 10 20 40 80 55 AA 33 01 02 U.3..... @.U.3..
20C0 04 08 10 20 40 80 55 AA 33 01 02 04 08 10 20 40 ... @.U.3..... @
20D0 80 55 AA 33 01 02 04 08 10 20 40 80 55 AA 33 01 .U.3..... @.U.3.
20E0 02 04 08 10 20 40 80 55 AA 33 01 02 04 08 10 20 .... @.U.3.....
20F0 40 80 55 AA 33 01 02 04 08 10 20 40 80 55 AA 33 @.U.3..... @.U.3
2100 01 02 04 08 10 20 40 80 55 AA 33 01 02 04 08 10 ..... @.U.3.....
*
```

Princípio de operação



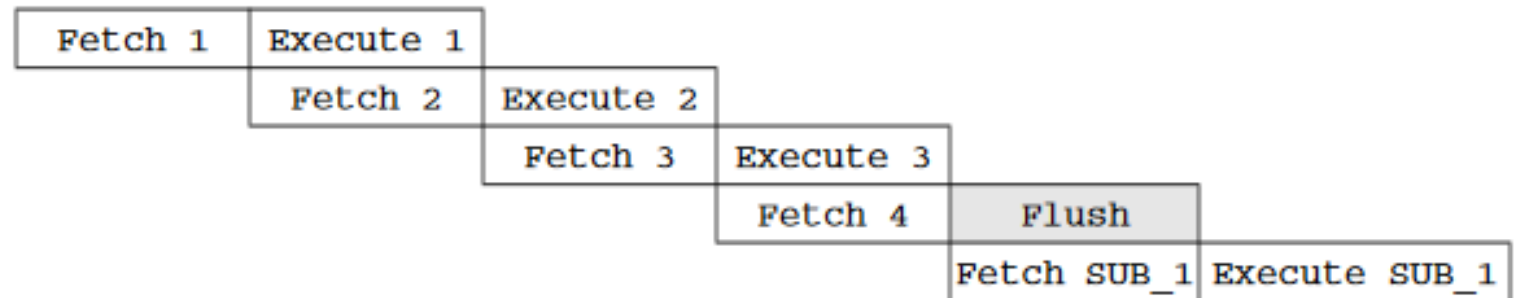
Ciclo de busca e execução

Exemplo de operação

PIC16F628A

EXAMPLE 3-1: INSTRUCTION PIPELINE FLOW

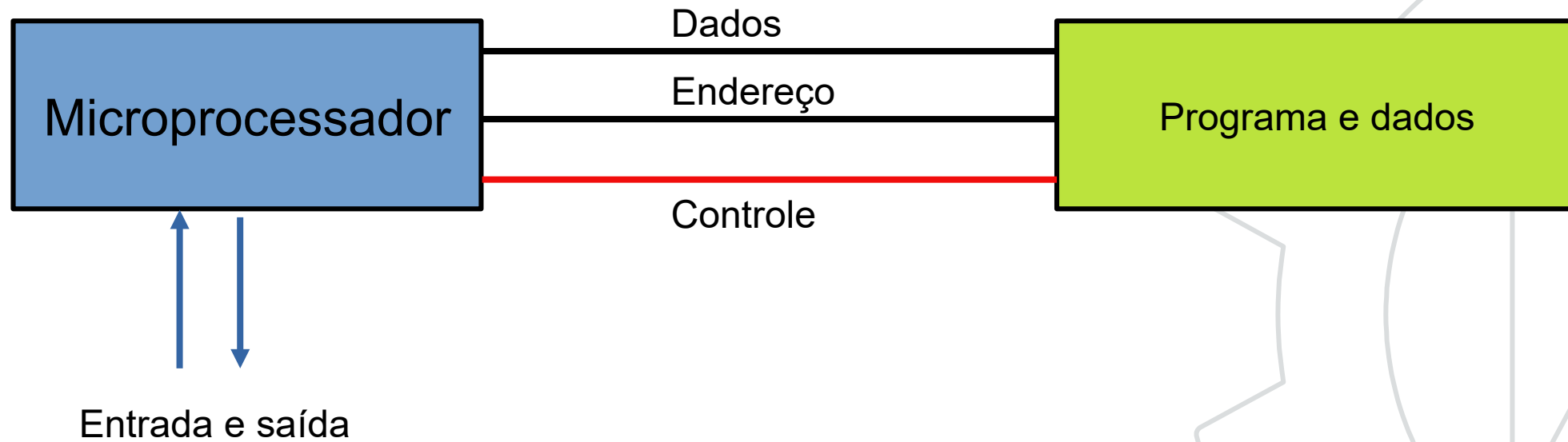
1. MOVLW 55h
2. MOVWF PORTB
3. CALL SUB_1
4. BSF PORTA, 3



Note: All instructions are single cycle except for any program branches. These take two cycles since the fetch instruction is “flushed” from the pipeline while the new instruction is being fetched and then executed.

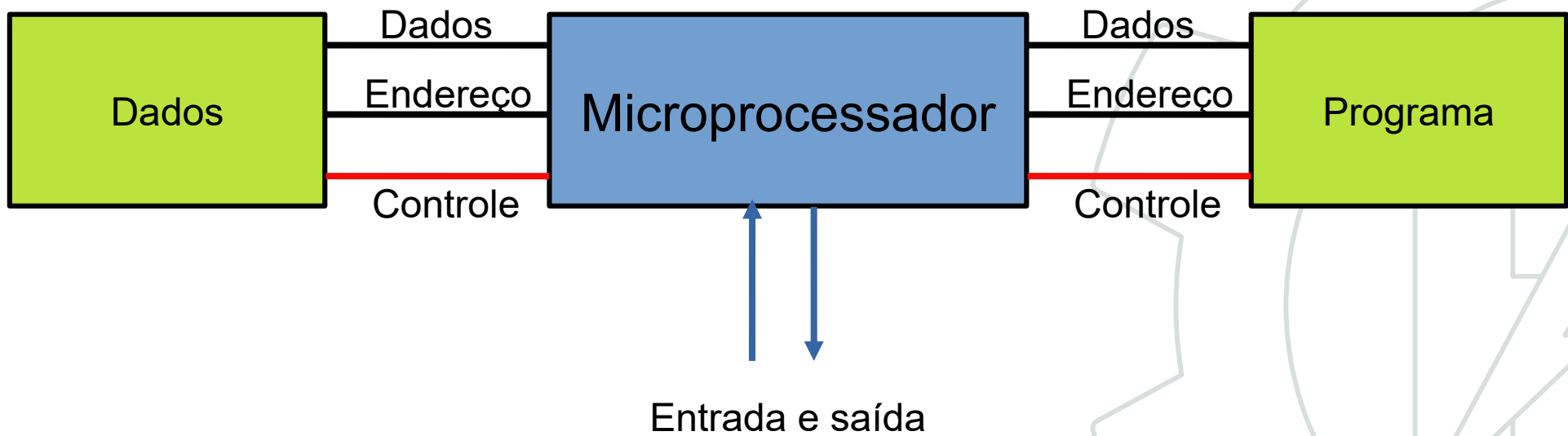
Uso da memória

Arquitetura de Von Neumann



Uso da memória

Arquitetura Harvard



Uso da memória

Arquitetura Harvard

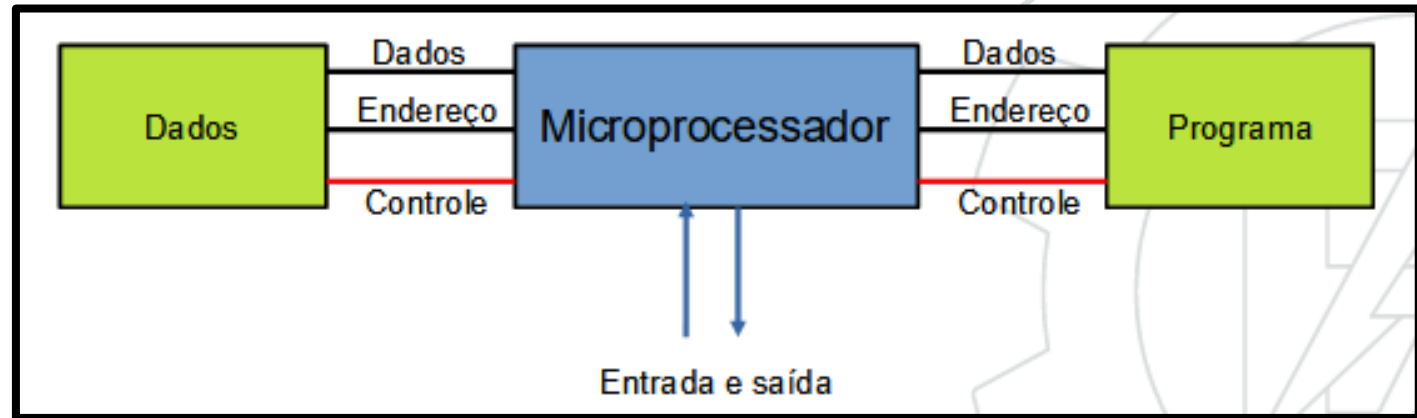
PIC16F628A

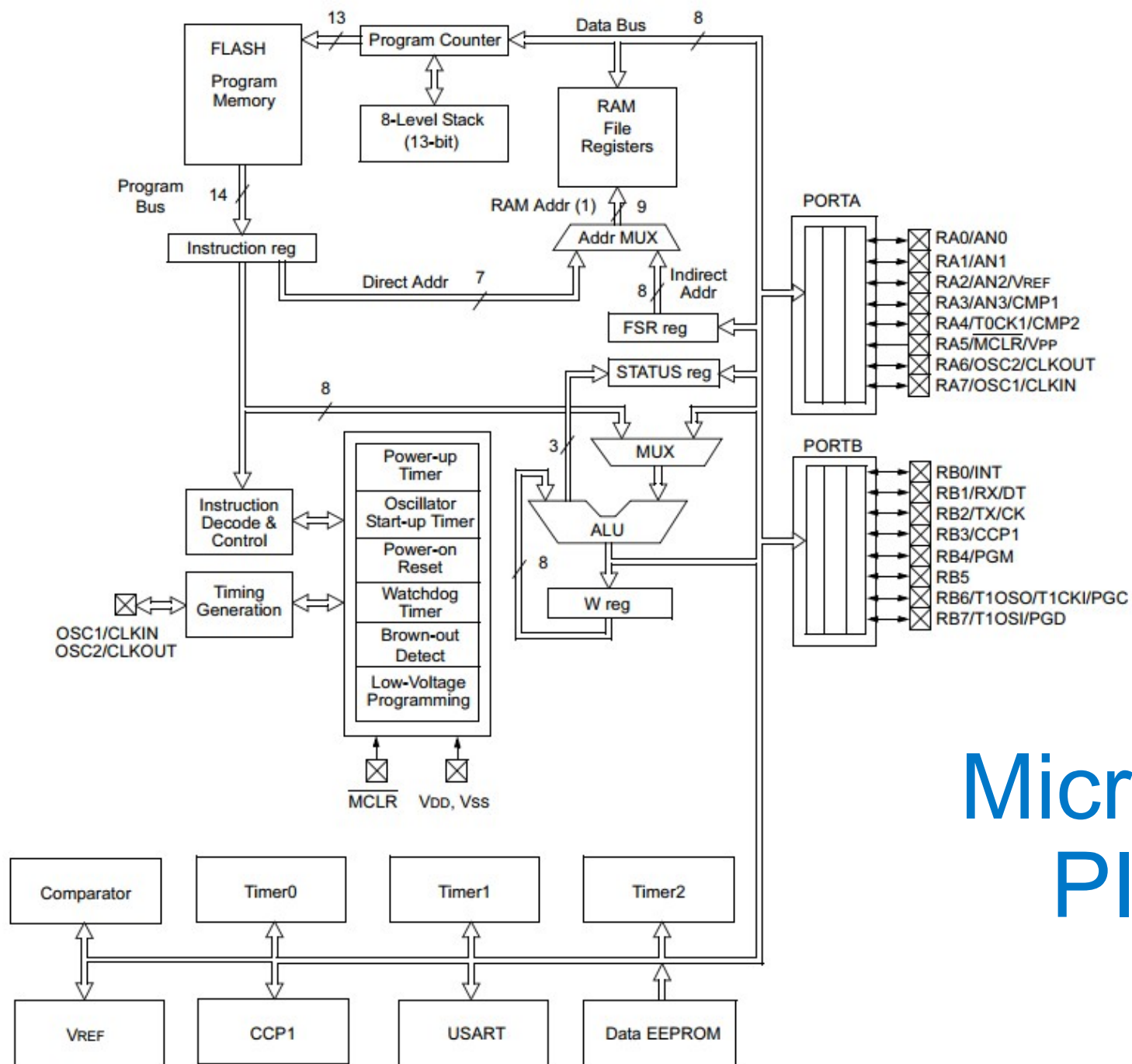
DEVICE MEMORY LIST

Device	Memory		
	Flash Program	RAM Data	EEPROM Data
PIC16F627A	1024 x 14	224 x 8	128 x 8
PIC16F628A	2048 x 14	224 x 8	128 x 8
PIC16F648A	4096 x 14	256 x 8	256 x 8
PIC16LF627A	1024 x 14	224 x 8	128 x 8
PIC16LF628A	2048 x 14	224 x 8	128 x 8
PIC16LF648A	4096 x 14	256 x 8	256 x 8



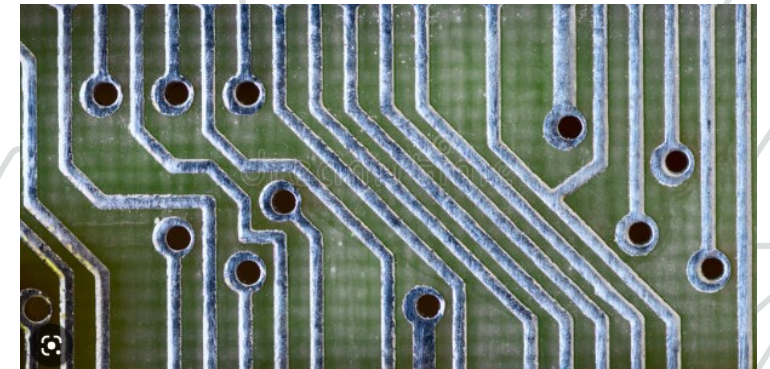
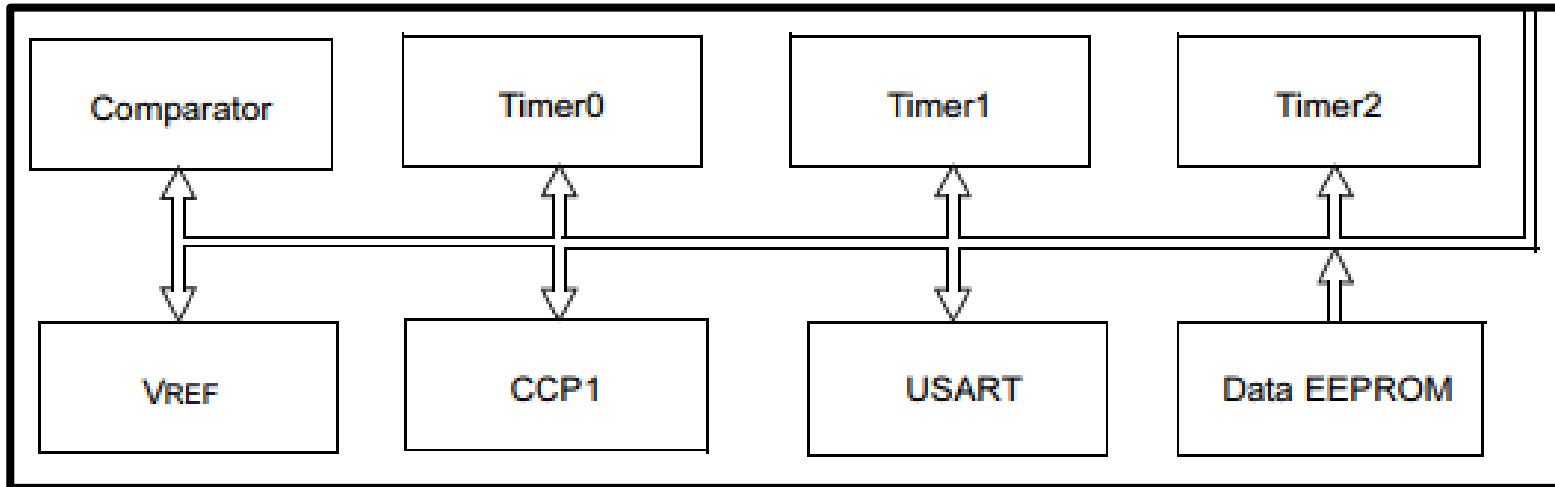
14 Bits





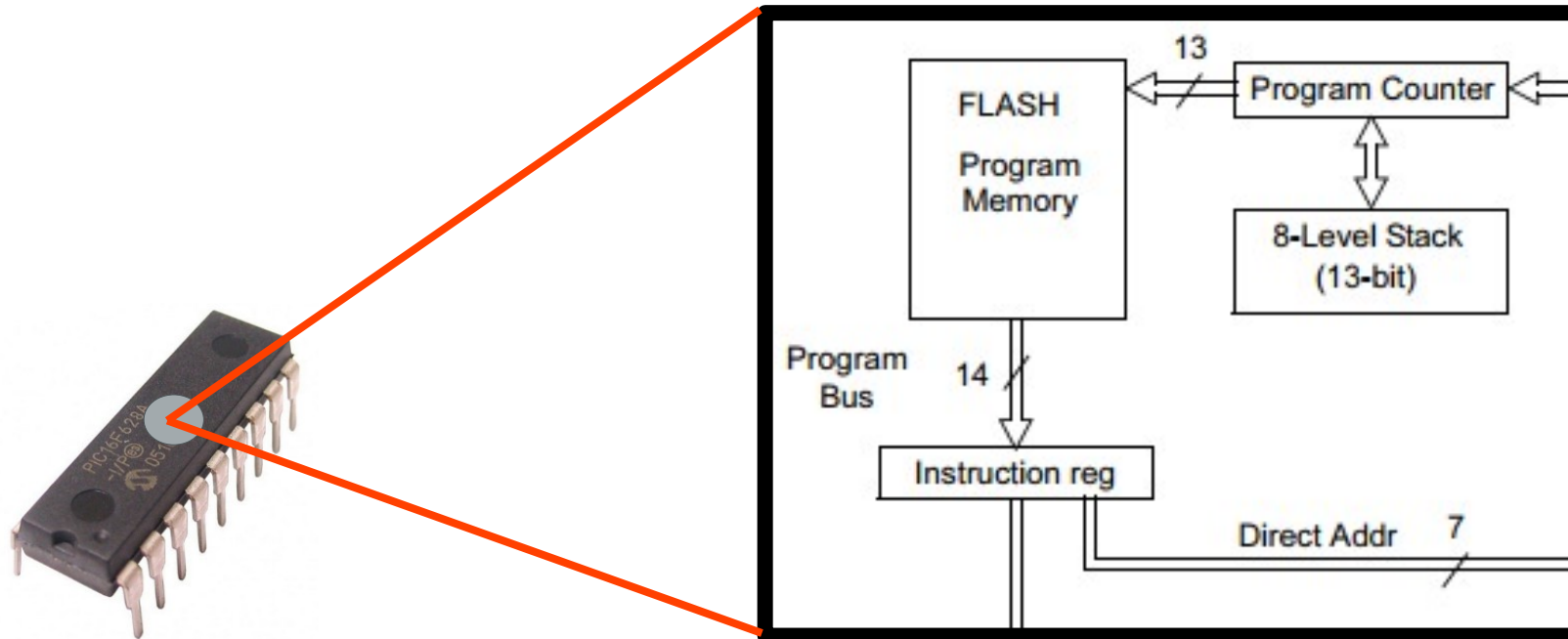
Microcontrolador PIC16F628A

Microcontrolador PIC16F628A



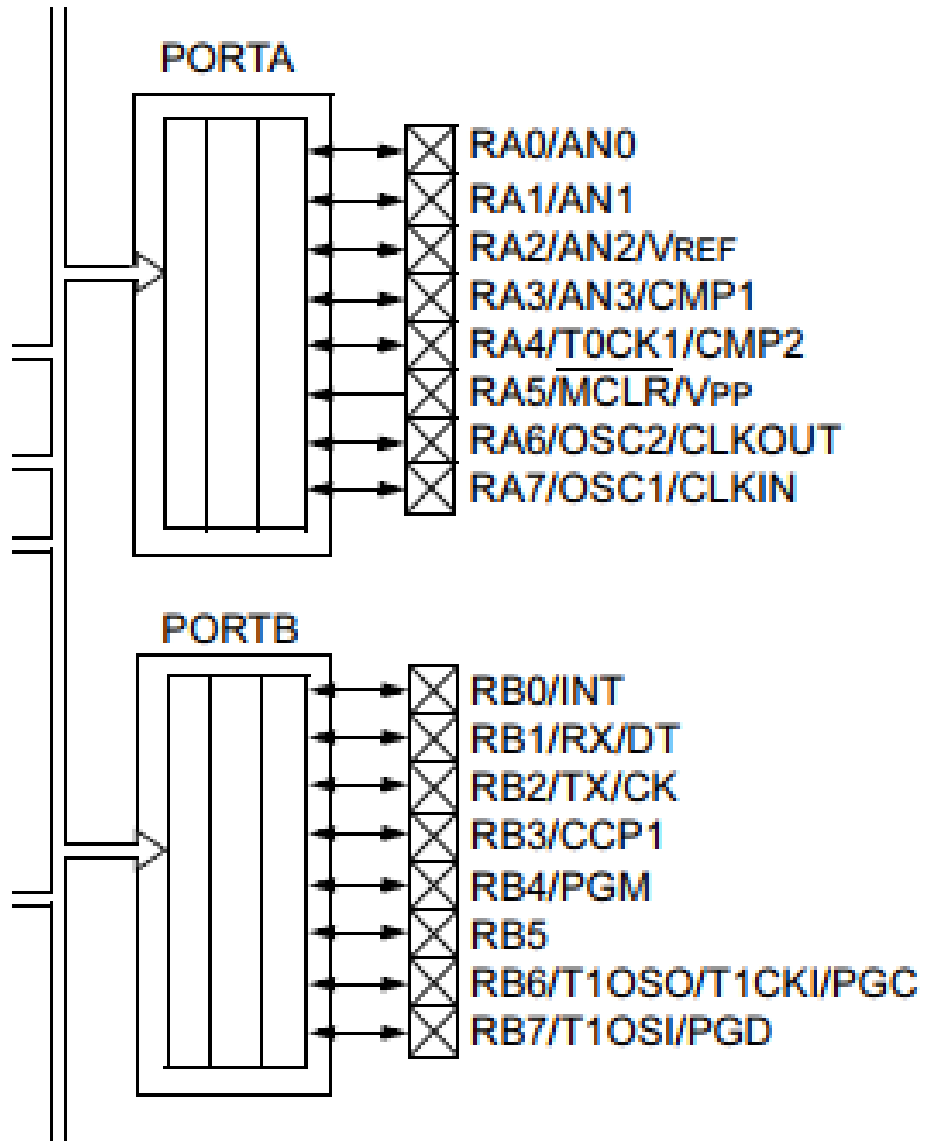
A comunicação com dispositivos é feita
por barramentos

Microcontrolador PIC16F628A



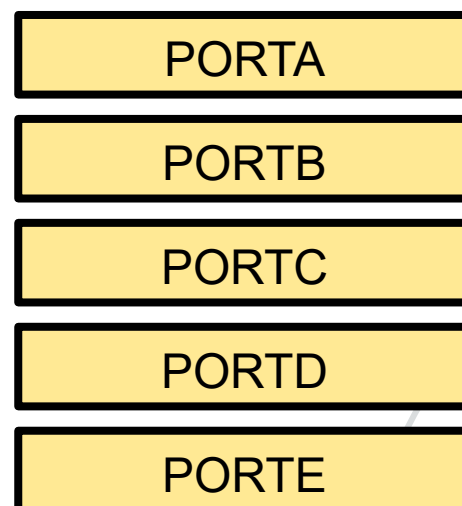
Cada fabricante e até mesmo cada processador utilizam estruturas internas diferentes

Microcontrolador PIC16F628A



PORTA

PORTB



4: RB3 is the alternate pin for CCP2 multiplexing.

PIC18f4520

Podemos ajustar um único bit ou ajustar todos em um único comando

PORTA

PORTB

PORTC

PORTD

PORTE

```
//liga/desliga cada led
```

```
BitSet(PORTD,0); //desliga o 1o led
```

```
BitSet(PORTD,1); //desliga o 2o led
```

```
BitClr(PORTD,2); //liga o 3o led
```

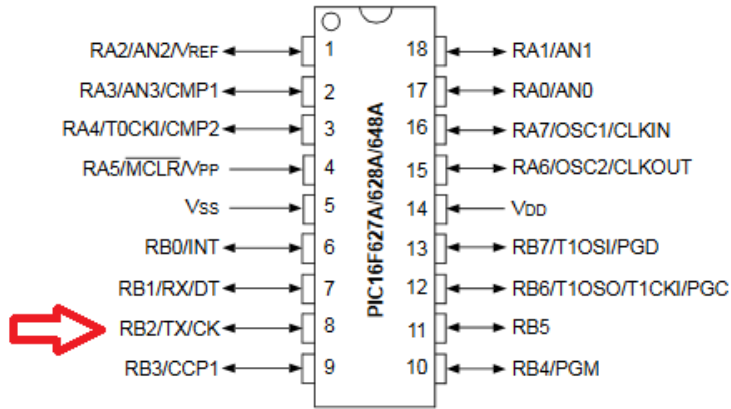
```
BitClr(PORTD,3); //liga o 4o led
```

```
//... continua até PORTD,7
```

```
PORTD = 0x77;
```

```
//mostra o número A no display
```


Pinos compartilhados



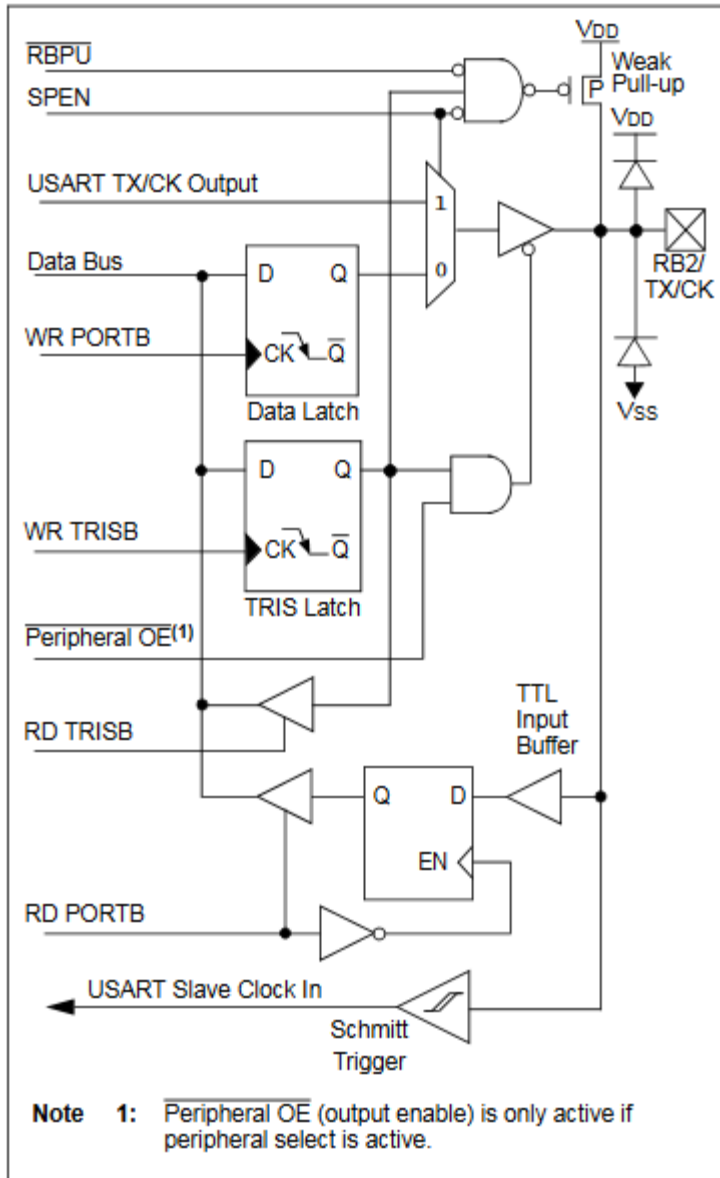
RA7/OSC1/CLKIN	RA7	ST	CMOS	Bidirectional I/O port
	OSC1	XTAL	—	Oscillator crystal input
	CLKIN	ST	—	External clock source input. RC biasing pin.
RB0/INT	RB0	TTL	CMOS	Bidirectional I/O port. Can be software programmed for internal weak pull-up.
	INT	ST	—	External interrupt
RB1/RX/DT	RB1	TTL	CMOS	Bidirectional I/O port. Can be software programmed for internal weak pull-up.
	RX	ST	—	USART receive pin
	DT	ST	CMOS	Synchronous data I/O
RB2/TX/CK	RB2	TTL	CMOS	Bidirectional I/O port. Can be software programmed for internal weak pull-up.
	TX	—	CMOS	USART transmit pin
	CK	ST	CMOS	Synchronous clock I/O
RB3/CCP1	RB3	TTL	CMOS	Bidirectional I/O port. Can be software programmed for internal weak pull-up.
	CCP1	ST	CMOS	Capture/Compare/PWM I/O

Legend: O = Output
 — = Not used
 TTL = TTL Input

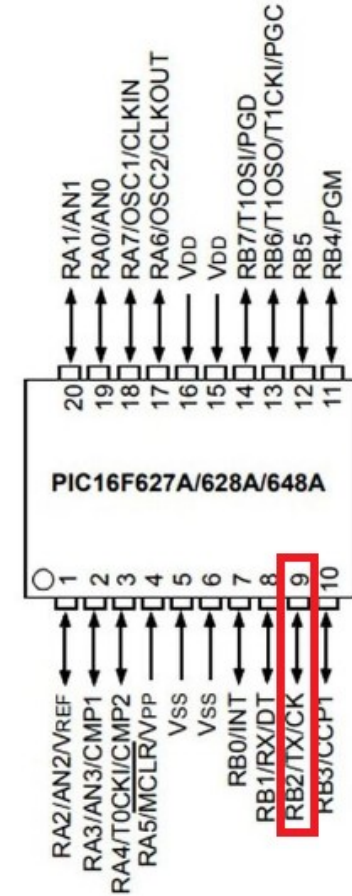
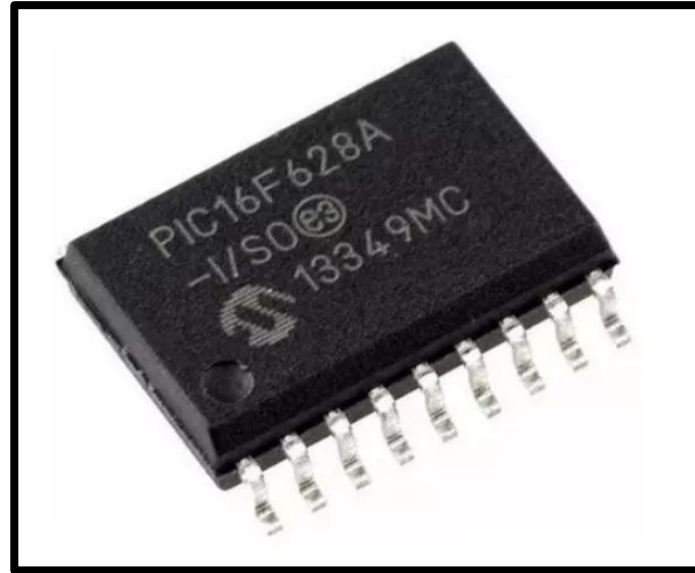
CMOS = CMOS Output
 I = Input
 OD = Open Drain Output

P = Power
 ST = Schmitt Trigger Input
 AN = Analog

BLOCK DIAGRAM OF RB2/TX/CK PIN



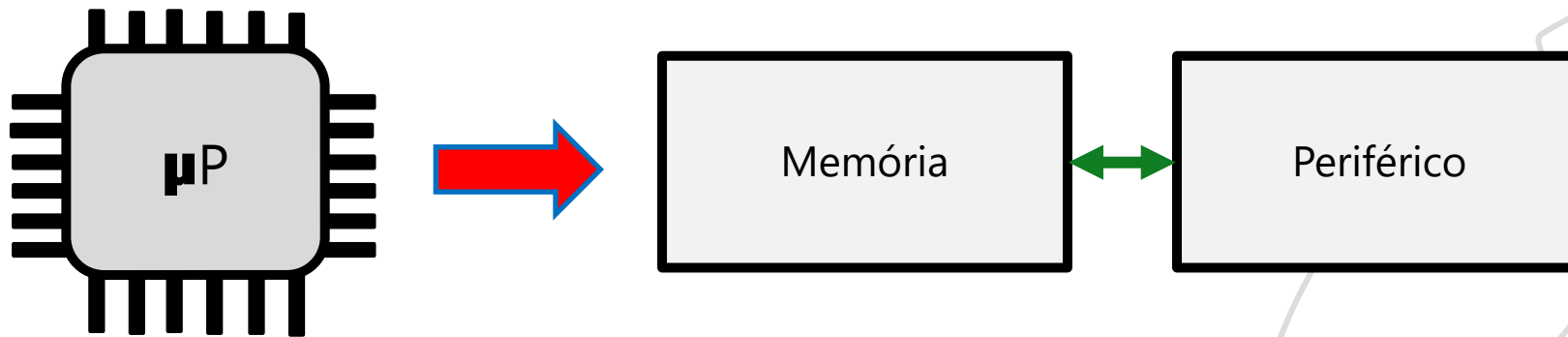
Pinos compartilhados



Interface com periféricos



Interface com periféricos



Os periféricos são controlados por valores colocados em memórias

DATA MEMORY MAP OF THE PIC16F627A AND PIC16F628A

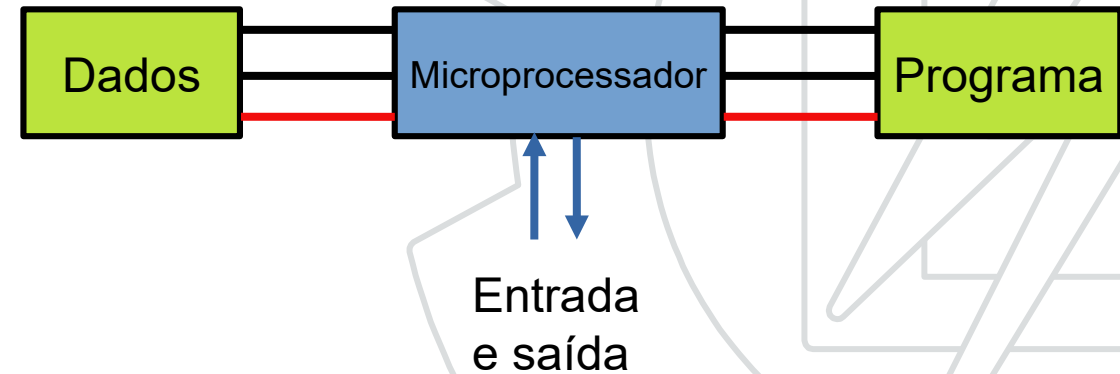
								File Address
Indirect addr. ⁽¹⁾	00h	Indirect addr. ⁽¹⁾	80h	Indirect addr. ⁽¹⁾	100h	Indirect addr. ⁽¹⁾	180h	
TMR0	01h	OPTION	81h	TMR0	101h	OPTION	181h	
PCL	02h	PCL	82h	PCL	102h	PCL	182h	
STATUS	03h	STATUS	83h	STATUS	103h	STATUS	183h	
FSR	04h	FSR	84h	FSR	104h	FSR	184h	
PORTA	05h	TRISA	85h		105h		185h	
PORTB	06h	TRISB	86h	PORTB	106h	TRISB	186h	
	07h		87h		107h		187h	
	08h		88h		108h		188h	
	09h		89h		109h		189h	
PCLATH	0Ah	PCLATH	8Ah	PCLATH	10Ah	PCLATH	18Ah	
INTCON	0Bh	INTCON	8Bh	INTCON	10Bh	INTCON	18Bh	
PIR1	0Ch	PIE1	8Ch		10Ch		18Ch	
	0Dh		8Dh		10Dh		18Dh	
TMR1L	0Eh	PCON	8Eh		10Eh		18Eh	
TMR1H	0Fh		8Fh		10Fh		18Fh	
T1CON	10h		90h					
TMR2	11h		91h					
T2CON	12h	PR2	92h					
	13h		93h					
	14h		94h					
CCPR1L	15h		95h					
CCPR1H	16h		96h					
CCP1CON	17h		97h					
RCSTA	18h	TXSTA	98h					
TXREG	19h	SPBRG	99h					
RCREG	1Ah	EEDATA	9Ah					
	1Bh	EEADR	9Bh					
	1Ch	EECON1	9Ch					
	1Dh	EECON2 ⁽¹⁾	9Dh					
	1Eh		9Eh					
CMCON	1Fh	VRCON	9Fh					
	20h		A0h	General Purpose Register 48 Bytes	11Fh			
General Purpose Register 80 Bytes		General Purpose Register 80 Bytes			120h			
					14Fh			
					150h			
	6Fh		EFh		16Fh			
16 Bytes	70h	accesses 70h-7Fh	F0h	accesses 70h-7Fh	170h	accesses 70h-7Fh	1EFh	
	7Fh		FFh		17Fh		1FFh	
Bank 0		Bank 1		Bank 2		Bank 3		

Unimplemented data memory locations, read as '0'.
Note 1: Not a physical register.

Mapa de memória

Lembrar que:

No Harvard temos o barramento de dados e o barramento de programa

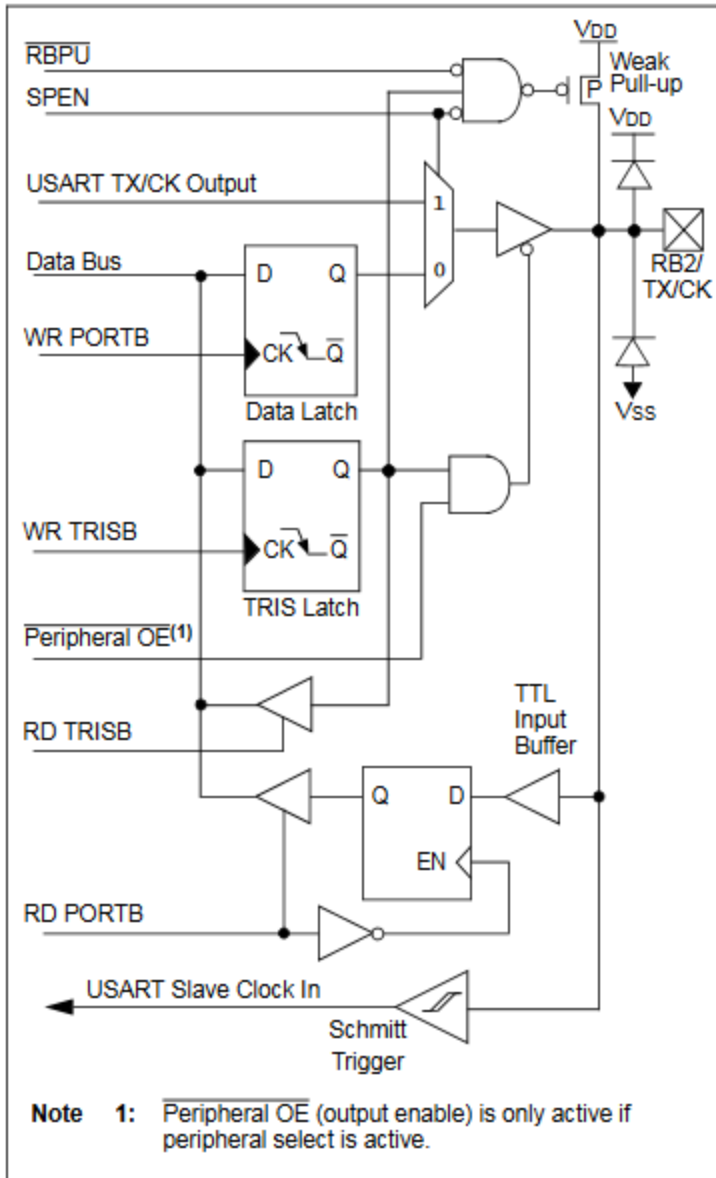


Exemplo: Pinos usados como saída

TABLE 4-3: SPECIAL REGISTERS SUMMARY BANK0

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Value on POR Reset ⁽¹⁾	Details on Page
Bank 0											
00h	INDF	Addressing this location uses contents of FSR to address data memory (not a physical register)								xxxx xxxx	30
01h	TMR0	Timer0 Module's Register								xxxx xxxx	47
02h	PCL	Program Counter's (PC) Least Significant Byte								0000 0000	30
03h	STATUS	IRP	RP1	RP0	$\overline{TO}$	$\overline{PD}$	Z	DC	C	0001 1xxx	24
04h	FSR	Indirect Data Memory Address Pointer								xxxx xxxx	30
05h	PORTA	RA7	RA6	RA5	RA4	RA3	RA2	RA1	RA0	xxxx 0000	33
06h	PORTB	RB7	RB6	RB5	RB4	RB3	RB2	RB1	RB0	xxxx xxxx	38
07h	—	Unimplemented								—	—
08h	—	Unimplemented								—	—
09h	—	Unimplemented								—	—
0Ah	PCLATH	—	—	—	Write Buffer for upper 5 bits of Program Counter					---0 0000	30
0Bh	INTCON	GIE	PEIE	TOIE	INTE	RBIE	TOIF	INTF	RBIF	0000 000x	26
0Ch	PIR1	EEIF	CMIF	RCIF	TXIF	—	CCP1IF	TMR2IF	TMR1IF	0000 -000	28

BLOCK DIAGRAM OF RB2/TX/CK PIN



Pinos compartilhados

SUMMARY OF REGISTERS ASSOCIATED WITH PORTB

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Value on POR	Value on All Other Resets
06h, 106h	PORTB	RB7	RB6	RB5	RB4 ⁽¹⁾	RB3	RB2	RB1	RB0	xxxx xxxx	uuuu uuuu
86h, 186h	TRISB	TRISB7	TRISB6	TRISB5	TRISB4	TRISB3	TRISB2	TRISB1	TRISB0	1111 1111	1111 1111
81h, 181h	OPTION	$\overline{\text{RBPU}}$	INTEDG	T0CS	T0SE	PSA	PS2	PS1	PS0	1111 1111	1111 1111

Legend: u = unchanged, x = unknown. Shaded cells are not used for PORTB.

Circuitos importantes



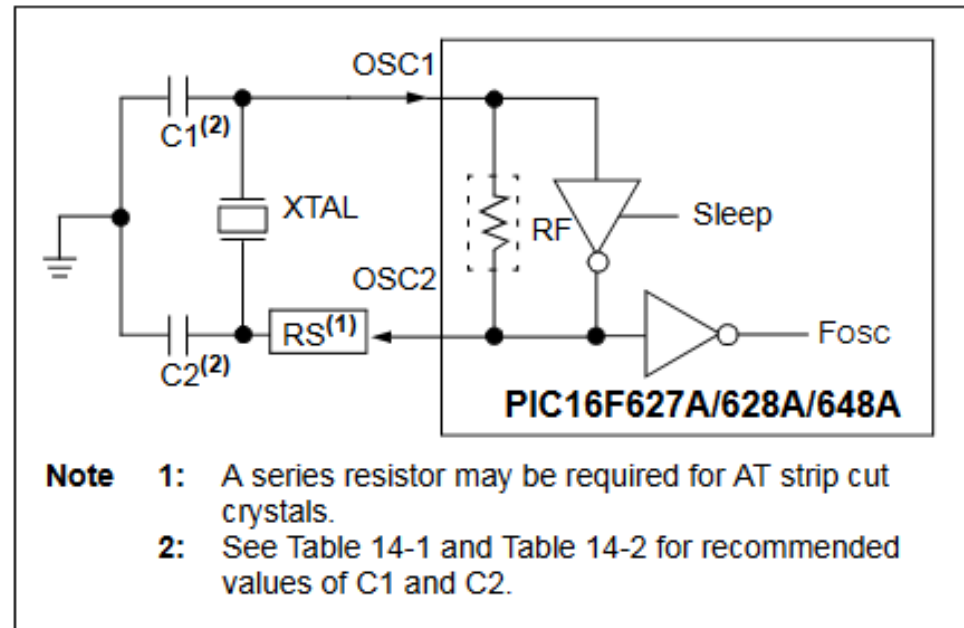
Circuitos importantes

- Alimentação
 - Depende do modelo do processador.
 - Geralmente existe uma faixa de tolerância.
- Clock
 - Cristal Externo (XT).
 - RC Externo.
 - RC Interno.
 - Fonte de clock externa.



Circuitos de Clock externo

CRYSTAL OPERATION (OR CERAMIC RESONATOR) (HS, XT OR LP OSC CONFIGURATION)

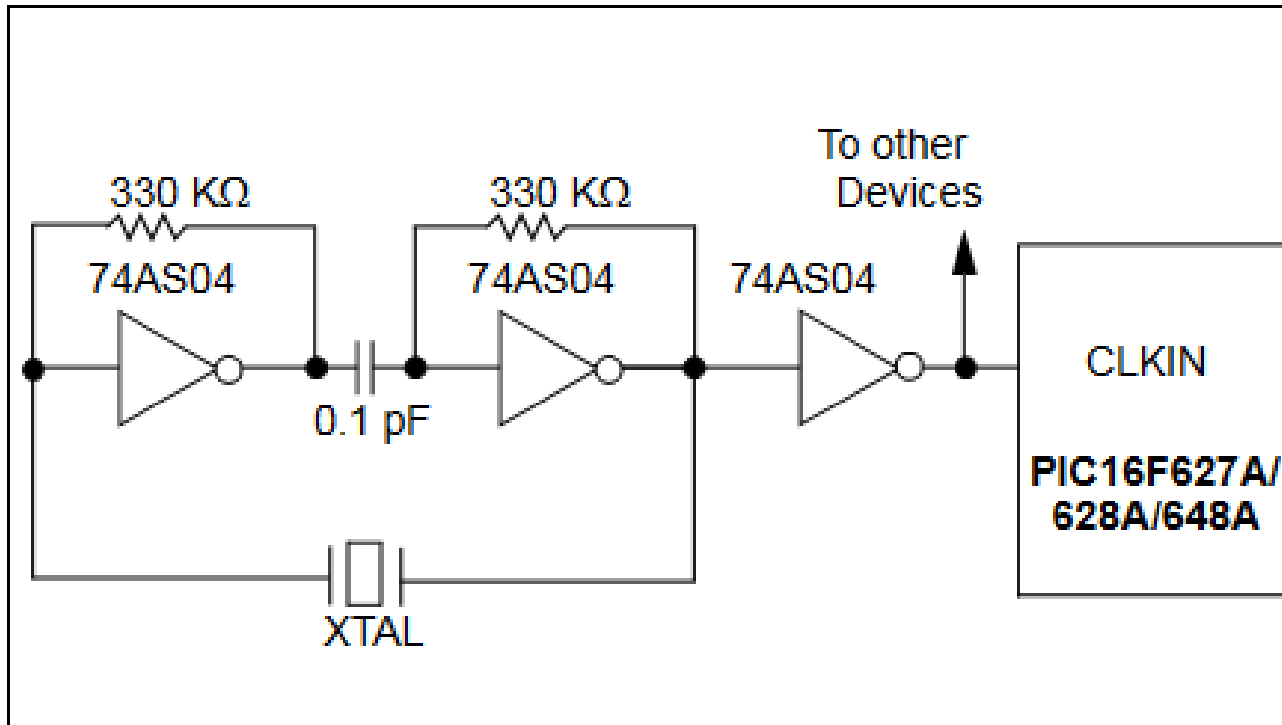


CAPACITOR SELECTION FOR CRYSTAL OSCILLATOR

Mode	Freq	OSC1(C1)	OSC2(C2)
LP	32 kHz	15-30 pF	15-30 pF
	200 kHz	0-15 pF	0-15 pF
XT	100 kHz	68-150 pF	150-200 pF
	2 MHz	15-30 pF	15-30 pF
	4 MHz	15-30 pF	15-30 pF
HS	8 MHz	15-30 pF	15-30 pF
	10 MHz	15-30 pF	15-30 pF
	20 MHz	15-30 pF	15-30 pF

Sinal de um gerador de clock

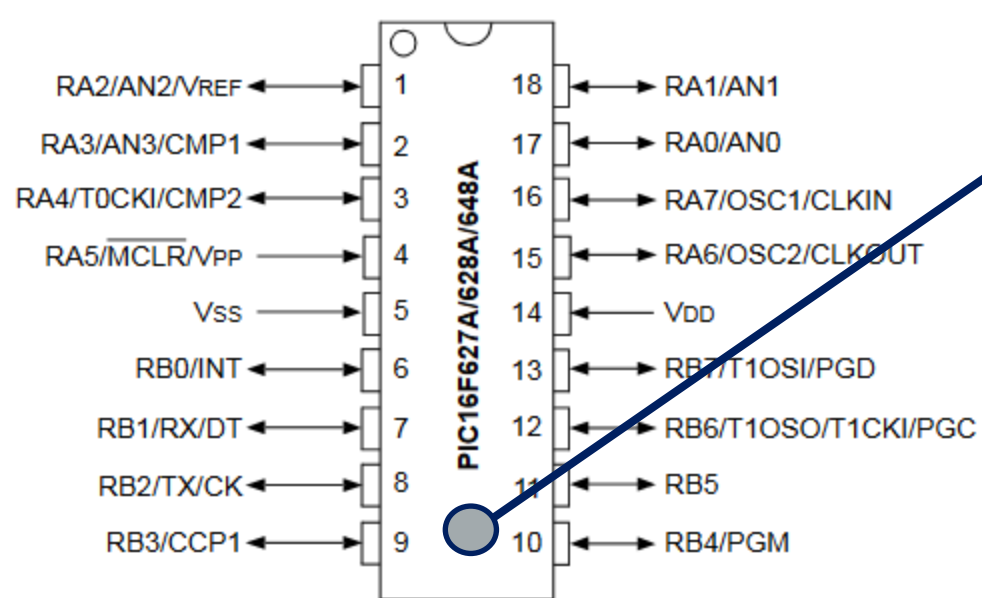
EXTERNAL SERIES RESONANT CRYSTAL OSCILLATOR CIRCUIT



Overclock



Clock interno



Configuração do hardware (#pragma)

```
// CONFIG
#pragma config FOSC = EXTRCCLK
#pragma config WDTE = ON
#pragma config PWRTE = OFF
#pragma config MCLRE = ON
#pragma config BOREN = ON
#pragma config LVP = ON
#pragma config CPD = OFF
#pragma config CP = OFF

void tempo(unsigned char atraso) {
    volatile unsigned char j, k;
    unsigned char loop;
    for (loop = 0; loop < atraso; loop++)
        for (j = 0; j < 41; j++)
            for (k = 0; k < 3; k++);
}
```



Até mais!

